

Take Home Assignment — Senior Full-Stack Developer

Candidate: Smit Vaghasiya | Deadline: 48 hours | Branch: feature/inventory-api | Submit: GitHub link only

Important — On Using AI Tools

You are free to use AI tools like Cursor, Copilot, or ChatGPT.

In the technical interview that follows, we will open your code and ask you to explain every decision.

File by file. Line by line if needed. If AI generated something you cannot explain — that is a failure.

Use AI to go faster. Not to replace understanding. The interview will verify the difference.

What this is

Build a role-based inventory management API. Multiple roles — Owner, Manager, Staff. Each role has different permissions. A background job processes low-stock alerts. Redis caches the most expensive queries.

We are testing RBAC schema design, Prisma depth, Redis caching strategy, BullMQ for async processing, and TypeScript discipline.

Rules

- Node.js + Express.js + TypeScript — strict mode, no any types.
- PostgreSQL + Prisma ORM.
- Redis for caching — correct TTL, correct invalidation.
- BullMQ for the low-stock alert job.
- Zod for all request validation.
- JWT auth with role embedded in the token.

Skip any of these and we stop reviewing.

Stack

Layer	What to use
Language	TypeScript strict mode
Framework	Express.js
ORM	Prisma + PostgreSQL
Cache	Redis
Queue	BullMQ
Validation	Zod
Auth	JWT

Roles and Permissions

Action	Owner	Manager	Staff
Create product	✓	✓	X
Update product	✓	✓	X
Delete product	✓	X	X

View all products	✓	✓	✓
Adjust stock	✓	✓	✓
View stock history	✓	✓	X
View low-stock alerts	✓	✓	X
Manage users	✓	X	X

RBAC must be enforced via middleware — not inside controller logic. A single `authorizeRoles(...roles)` middleware handles all permission checks.

What to Build

Module 1 — Auth

Endpoint	Access	What it does
POST /auth/register	Public	Register with name, email, password, role (owner manager staff). Hash password. Return JWT with role.
POST /auth/login	Public	Login. Return JWT.

Module 2 — Products

Endpoint	Access	What it does
POST /products	Owner, Manager	Create product with name, description, price (Int rupees), stock, lowStockThreshold.
GET /products	All roles	List all products with current stock. Cache in Redis — 5 minute TTL. Invalidate on any stock change.
GET /products/:id	All roles	Single product. Include stock history summary.
PUT /products/:id	Owner, Manager	Update product details. Invalidate cache.
DELETE /products/:id	Owner only	Delete product. Clear cache.

Module 3 — Stock Management

Endpoint	Access	What it does
POST /products/:id/stock	All roles	Adjust stock. Accepts quantity (positive = restock, negative = sale) and reason (sale return restock damage). Creates a StockHistory record. If new stock is below lowStockThreshold, enqueue a low-stock alert job. Use Prisma transaction — stock update and history record are atomic.
GET /products/:id/stock/history	Owner, Manager	Get full stock history for a product — date, quantity change, reason, who made the change.

Module 4 — Low Stock Alert Job

When stock drops below lowStockThreshold, a BullMQ job is enqueued. The worker processes the alert — log which product is low, current stock, threshold. The API does not wait for this — it returns immediately after enqueueing.

- Queue name: low-stock-alerts
- Job payload: productId, productName, currentStock, threshold.
- Worker logs the alert — no actual email or SMS needed.
- If the same product triggers another alert within 24 hours — skip it (deduplication).

Schema

```
User          - id, name, email, password (hashed), role, createdAt
Product       - id, name, description, price (Int), stock, lowStockThreshold,
               createdAt, updatedAt
StockHistory  - id, productId (FK), userId (FK), quantityChange, reason, stockAfter,
               createdAt
AlertLog      - id, productId (FK), triggeredAt, createdAt
```

The Cache Invalidation Challenge

GET /products is cached with a 5-minute TTL. When stock changes on any product — the cache must be invalidated so the next read is fresh. Think carefully about when to invalidate and what key to delete.

In the technical interview we will specifically ask: what happens if stock is updated and a user reads the product list 30 seconds later? Walk through your code.

Bonus

- Rate limiting on POST /auth/login.
- GitHub Actions tsc --noEmit on push.
- Jest test for RBAC — Staff trying to delete a product should get 403.

Instant Rejection

- RBAC enforced inside controllers not middleware.
- No Prisma transaction on stock adjustment.
- BullMQ not used — alert processed synchronously.
- any TypeScript type anywhere.
- Single commit.
- Zip submission.
- App does not run.

Git and Submission

- Branch named feature/inventory-api.
- Open a PR to main. In the PR description explain your key architectural decisions.
- Minimum 8 meaningful commits — show your work progressing in stages.
- Commit messages that describe what you built — not 'update' or 'fix'.
- README — setup instructions, env vars needed, how to run locally.
- .env.example committed with empty values.
- Repo must be public before you send the link.

Send the GitHub link to your HR contact within 48 hours. Questions about requirements — ask before you start, not after you submit.

